RACEDAY – PART 1
SYSTEM PLANNING AND DATABASE

Portfolio of Evidence (PoE)

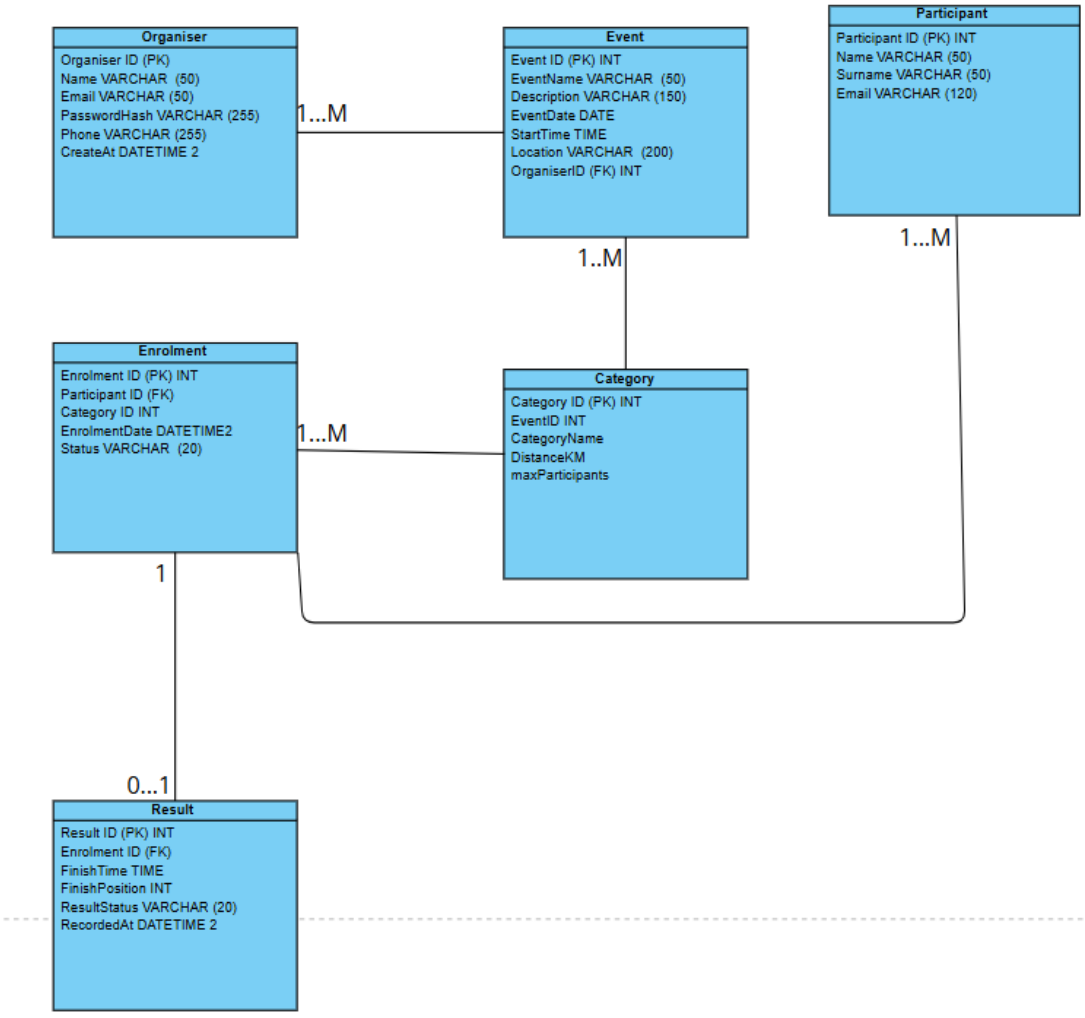
Student Name: _Tshepang Mateba
Student Number: 10482201
Module: _Programming 2B
Date: _4 September 2026

1. Introduction

RaceDay is a full-stack web-based event management system designed for the South African road running, walking and cycling community. Part 1 focuses on planning the relational database and RESTful API before application code is implemented. The design supports two roles: Organiser and Participant.

2. Section A – Entity Relationship Diagram (ERD)

The ERD contains six entities: Organiser, Participant, Event, Category, Enrolment and Result. Primary keys are marked PK and foreign keys are marked FK. The relationships support event management, category selection, participant enrolment and recording of race results.



3. Entity Summary

Entity	Purpose
Organiser	Stores users who create and manage RaceDay events.
Participant	Stores users who browse events, enrol and view personal results.
Event	Stores the main details of a race, walk or cycling event and its organiser.
Category	Stores the participation categories available within an event.
Enrolment	Associates a participant with a selected event category.
Result	Stores the finishing result associated with an enrolment.

4. Section B – API Endpoint Plan

The following endpoints cover authentication, user profiles, events, categories, enrolments and results. Role-based access is enforced at API level. The implemented API in Part 2 should be kept consistent with this plan unless a documented change is required.

HTTP	Route	Description	Role	Request body	Expected response
POST	/api/auth/register	Creates a participant account.	Public	{"firstName":"...","lastName":"...","email":"...","password":"...","phone":"..."}	201 Created; 400 Bad Request if validation fails; 409 Conflict if email exists.
POST	/api/auth/login	Authenticates a user and returns an access token and role.	Public	{"email":"...","password":"..."}	200 OK with token and role; 401 Unauthorized for invalid credentials.
GET	/api/users/me	Returns the authenticated user's profile.	Any (logged in)	None	200 OK with profile; 401 Unauthorized if not authenticated.
PUT	/api/users/me	Updates the authenticated user's profile.	Any (logged in)	{"firstName":"...","lastName":"...","phone":"..."}	200 OK with updated profile; 400 Bad Request for invalid data.

GET	/api/events	Returns upcoming events, optionally filtered by date/location.	Public	None	200 OK with event list.
GET	/api/events/{id}	Returns one event with its categories and basic event information.	Public	None	200 OK; 404 Not Found if event does not exist.
POST	/api/events	Creates a new event.	Organiser	{"eventName":"...","description":"...","eventDate":"YYYY-MM-DD","startTime":"HH:MM","location":"...","distanceKm":21.1}	201 Created; 400 Bad Request for invalid data; 403 Forbidden for non-organiser.
PUT	/api/events/{id}	Updates an event owned by the organiser.	Organiser	{"eventName":"...","description":"...","eventDate":"YYYY-MM-DD","startTime":"HH:MM","location":"...","distanceKm":21.1}	200 OK; 404 Not Found; 403 Forbidden.
DELETE	/api/events/{id}	Deletes an event owned by the organiser.	Organiser	None	204 No Content; 404 Not Found; 403 Forbidden.
GET	/api/categories	Returns available event categories.	Public	None	200 OK with category list.
GET	/api/events/{eventId}/categories	Returns categories belonging to a specific event.	Public	None	200 OK; 404 Not Found if event does not exist.
POST	/api/events/{eventId}/categories	Creates a category for an event.	Organiser	{"categoryName":"...","distanceKm":10.0,"maximumParticipants":500,	201 Created; 400 Bad Request; 403 Forbidden;

				"entryFee":150.00}	404 Not Found.
PUT	/api/categories/{id}	Updates an event category.	Organiser	{"categoryName":"...","distanceKm":10.0,"maximumParticipants":500,"entryFee":150.00}	200 OK; 404 Not Found; 403 Forbidden.
DELETE	/api/categories/{id}	Deletes an event category when it has no enrolments.	Organiser	None	204 No Content; 404 Not Found; 409 Conflict if in use.
POST	/api/events/{eventId}/enrolments	Enrols the logged-in participant in an event category.	Participant	{"categoryId":1}	201 Created; 400 Bad Request; 404 Not Found; 409 Conflict if already enrolled/full.
GET	/api/users/me/enrolments	Returns the logged-in participant's enrolments.	Participant	None	200 OK with enrolment list.
GET	/api/events/{eventId}/enrolments	Returns all enrolments for an event.	Organiser	None	200 OK; 403 Forbidden; 404 Not Found.
DELETE	/api/enrolments/{id}	Cancels the logged-in participant's enrolment.	Participant	None	204 No Content; 404 Not Found; 409 Conflict if cancellation is not allowed.
POST	/api/events/{eventId}/results	Captures a participant result for an event.	Organiser	{"enrolmentId":1,"finishTime":"01:48:32","finishPosition":12,"resultStatus":"Finished"}	201 Created; 400 Bad Request; 404 Not Found; 403 Forbidden.
PUT	/api/results/{id}	Corrects an existing result.	Organiser	{"finishTime":"01:47:59","finishPosition":11,"resultStatus":"Finished"}	200 OK; 404 Not Found; 403 Forbidden.

GET	/api/users/me/results	Returns the logged-in participant's personal results history.	Participant	None	200 OK with result history.
GET	/api/events/{eventId}/results	Returns results for an event.	Organiser	None	200 OK; 403 Forbidden; 404 Not Found.

5. Section C – SQL Database Script

The SQL Server script creates the RaceDayDB database and six tables matching the ERD. It defines primary keys, foreign keys, NOT NULL, UNIQUE, DEFAULT and CHECK constraints. It also seeds two organisers, two participants, three events, categories for every event, sample enrolments and sample results.

6. Database Design Decisions

Organiser and Participant are separate entities so role-specific data and permissions can be managed clearly.
Category belongs to Event, allowing each event to offer multiple participation options.
Enrolment is used to resolve the participant-to-category relationship and records the participant's selected category.
Result references Enrolment, ensuring a result is associated with a specific participant's registration.
Unique email addresses prevent duplicate accounts within each role.
Status fields use CHECK constraints to limit invalid values.
Seed data is included so the database can be tested immediately in SQL Server Management Studio.

7. Submission Files

RaceDay_ERD.png
API_Endpoint_Plan.md
RaceDayDB.sql